

一、单项选择题

1.B 2.D 3.D 4.C 5.C 6.A 7.C 8.A 9.C 10.A
11.B 12.B 13.B 14.D 15.D

二、判断题

1.× 2.× 3.× (还需要释放结点所占的内存空间)
4.× (只有中序遍历才会得到有序序列)
5.× 6.√ 7.× 8.× 9.× 10.√ 11.× 12.√ 13.× 14.× 15.×

三、填空题

1. 栈
2. 循环队列
3. 数据的运算
4. $(n+1)/2$
5. 计算出以 V_i 为头结点的边表结点的个数
6. 稀疏
7. 队列
8. 输入序列有序、 平衡二叉树
9. 链式
10. 路径长度递增
11. 3
12. 中序序列下的后继结点的地址
13. 一维数组、 完全二叉树

四、简答题

1. 在待排序列表 $L[1...n]$ 中任取一个元素做为基准(通常取序列中的第一个关键字), 通过一趟排序将待排序列表划分为独立的两部分 $L[1...k-1]$ 和 $L[k+1...n]$, 使得 $L[1...k-1]$ 中的所有元素小于基准, $L[k+1...n]$ 中的所有元素大于等于基准, 则基准关键字在这一趟排序过程中放在了其最终位置 $L[k]$ 上, 这个过程称为一趟快速排序。而后分别递归地对两个表重复上述过程, 直至每部分内只有一个元素或空为止, 即所有元素都放在了其最终位置上。

当初始序列基本有序或基本逆序时, 快排序效率最低, 其最坏情况下的时间复杂度为 $O(n^2)$ 。

【注】个人觉得上面这段话不够简洁, 建议还是自己理解再总结一下, 这样记着也方便

2. 由散列函数计算出各关键字对应的散列地址如下表所示:

关键字	23	12	8	27	32	70	16	19	49
地址	1	1	8	5	10	4	5	8	5

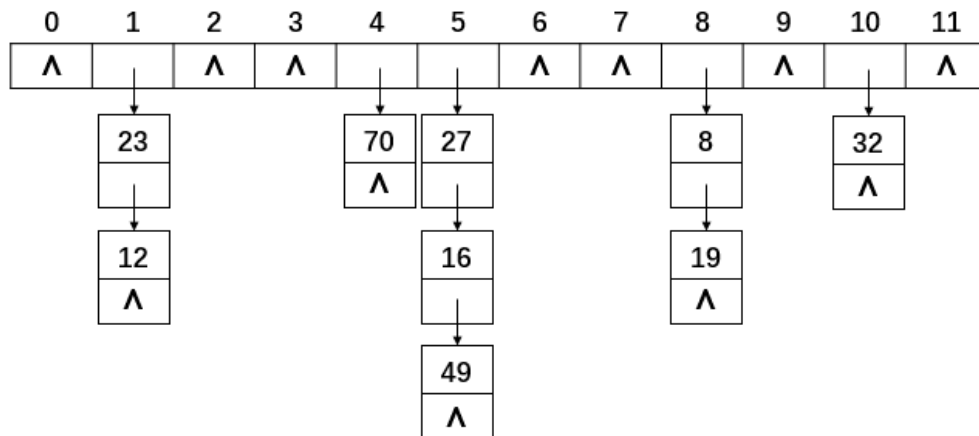
①由此构造出以线性探测解决冲突的哈希表为:

地址	0	1	2	3	4	5	6	7	8	9	10	11
关键字		23	12		70	27	16	49	8	19	32	

$$ASL_{成功} = (1+2+1+1+2+3+1+2+1)/9 = 14/9$$

$$ASL_{失败} = (1+3+2+1+8+7+6+5+4+3+2)/11 = 42/11$$

②链地址法处理冲突的哈希表为

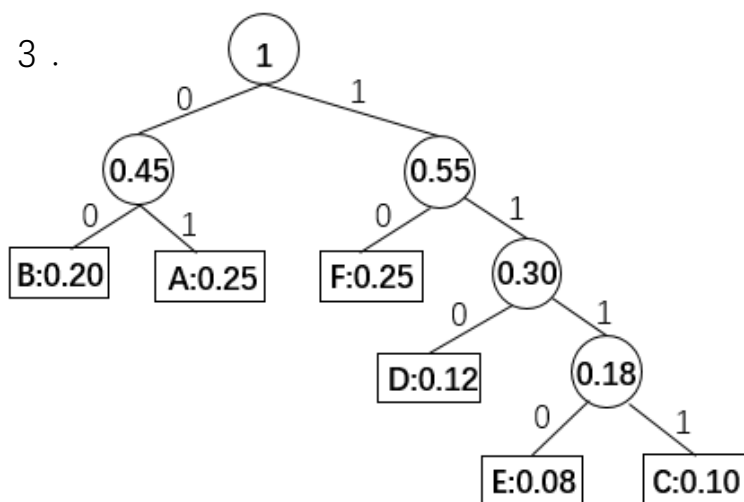


$$ASL_1 = (1+2+1+1+2+3+1+2+1)/9 = 14/9$$

$$ASL_2 = (1+3+1+1+2+4+1+1+3+1+2)/11 = 20/11$$

【注】对于链地址法的查找失败次数，有两种说法，其一认为比较到空结点才算失败，所以比较次数等于冲突次数加一，其二认为只有与关键字的比较才算比较次数。去年经过讨论选择了第一种说法，

3 .



哈夫曼编码

B	00
A	01
F	10
D	110
E	1110
C	1111

4 . 当待排序序列中有两个或两个以上的相同关键字时，排序前和排序后这些关键字的相对位置有没有发生变化。如果未发生变化就说明是稳定的排序，否则就是不稳定的。

简单选择排序是不稳定的，因为在第 i 趟找到最小元素后和第 i 个元素交换，可能会导致第 i 个元素与其有相同值的元素的相对位置发

生变化。

堆排序是不稳定的，因为它在排序调整时有可能把后面相同值的元素调整到前面

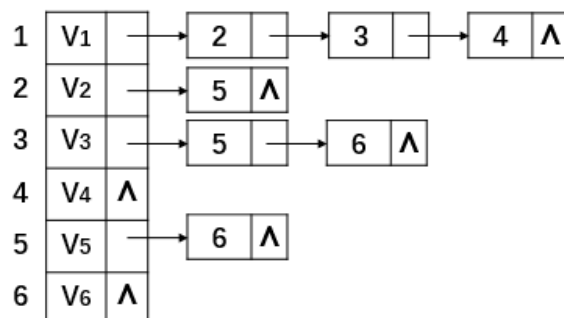
快速排序是不稳定的，当右端区间有两个关键字相同，且均小于基准值，则在交换到左端区间后，它的相对位置会发生变化。

希尔排序是不稳定的，当相同关键字被划分到了不同子序列时它们的相对位置可能会发生改变。

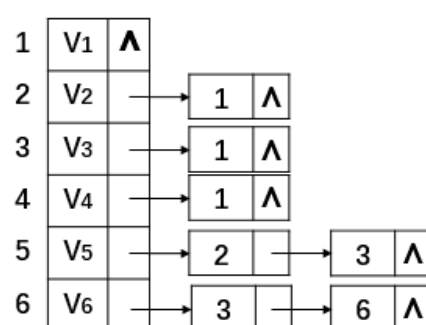
【注】天勤上有个助记，我觉得挺好用的：考研复习痛苦啊，情绪不稳定，快些(希)选一堆好友来 Timi 吧

5.

邻接表



逆邻接表



6.

	V1	V2	V3	V4	V5	V6
V1	∞	100	30	20	∞	∞
V2	∞	∞	∞	∞	∞	60
V3	∞	∞	∞	∞	50	∞
V4	∞	∞	∞	∞	∞	50
V5	∞	∞	∞	∞	∞	40
V6	∞	∞	∞	∞	∞	∞

从V1到各顶点的dist值和最短路径的求解过程

顶点	第1轮	第2轮	第3轮	第4轮	第5轮
V ₂	100 V ₁ →V ₂	100 V ₁ →V ₂	100 V ₁ →V ₂	100 V ₁ →V ₂	100 V ₁ →V ₂
V ₃	30 V ₁ →V ₃	30 V ₁ →V ₃			
V ₄	20 V ₁ →V ₄				
V ₅	∞	∞	80 V ₁ →V ₃ →V ₅	80 V ₁ →V ₃ →V ₅	
V ₆	∞	70 V ₁ →V ₄ →V ₆	70 V ₁ →V ₄ →V ₆		
集合S	{V ₁ ,V ₄ }	{V ₁ ,V ₄ ,V ₃ }	{V ₁ ,V ₄ ,V ₃ ,V ₆ }	{V ₁ ,V ₄ ,V ₃ ,V ₆ , V ₅ }	{V ₁ ,V ₄ ,V ₃ ,V ₆ , V ₅ , V ₂ }

五、算法设计题

```

1. 1) void InsertSort(Sqlist &L)
    {
        int i, j;
        int temp;
        for(i = 1; i < L.length; ++i)
        {
            temp = L.s[i]; //将待插入关键字暂存于temp中
            j = i - 1;
            //从待排关键字之前的关键字开始扫描, 如果大于待排关键字, 则后移一位
            while(j >= 0 && temp.key < L.s[j].key)
            {
                L.s[j+1] = L.s[j];
                --j;
            }
            L.s[j+1] = temp; //找到插入位置, 将temp中暂存的待排关键字插入
        }
    }

```

```

2) int BinSearch(Sqlist L, int k, StudentType &retult)
    {
        int low = 0, high = L.length-1, mid;
        while(low <= high)
        {
            mid = (low+high)/2;
            if(L.s[mid].key == k)
            {
                result = L.s[mid];
                return 1;
            }
            else if(L.s[mid].key > k) high=mid-1;
            else low=mid+1;
        }
        return 0;
    }

```

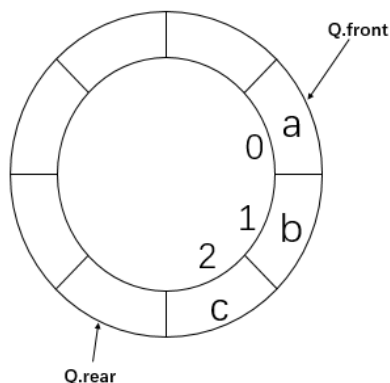
2. 1)

```
void count(Bstree* t)
{
    int n1, n2;
    if(t == NULL) return 0;
    else
    {
        n1 = count(t->lchild);
        n2 = count(t->rchild);
        return n1+n2+1;
    }
}
```

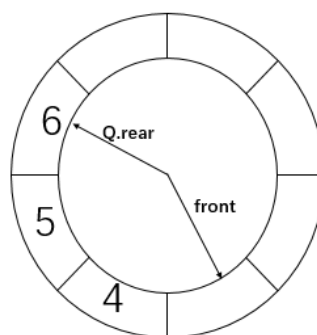
2)

```
BStree* Search(BStree *t, int key)
{
    if(t != NULL)
    {
        if(t->data == key) return t;
        Search(t->lchild, key);
        Search(t->rchild, key);
    }
    return NULL;
}
```

3. 王道书和天勤书上对于队列有稍微的不同，但自己理解就好，不用太扣细节，此处推荐用王道书上的定义



王道书上是尾指针指向空，先放进元素再移动指针，先出队列再移动指针



天勤上是队首指针指向空，需要移动指针再出队列，先移动指针再进入元素

```
int EnQueue(SqQueue &Q, ElemType x)
{
    if((Q.rear+1)%MAXSIZE == Q.front) return 0;
    Q.Q[Q.rear+1]= x;
    Q.rear = (Q.rear+1)%MAXSIZE;
    return 1;
}

int DeQueue(SqQueue &Q, ElemType &x)
{
    if(Q.rear == Q.front) return 0;
    x = Q.Q[Q.front];
    Q.front = (Q.front+1)%MAXSIZE;
    return 1;
}
```